CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

ESCOLA SUPERIOR POLITÉCNICA

CURSO DE DESENVOLVIMENTO WEB BACK-END

ATIVIDADE PRÁTICA

API REST – BAOZI STORE

Desenvolvimento com Java, Spring Boot, Spring Data JPA e H2 Database

Murilo Bauermann Cremm Domingues Ferreira – RU: 4425054

Professora: Luciane Kanashiro, Me.

Cabreúva – SP

2026

1. Descrição da Situação Fictícia

A Baozi Store é uma pequena loja que vende pão chinês artesanal. Para melhorar a organização do negócio e agilizar o atendimento, foi desenvolvido um sistema simples e moderno para controlar de forma integrada os clientes, produtos e pedidos através de uma API REST.

Um cliente chamado **Murilo4425054** realizou seu cadastro com sucesso no sistema. O produto vendido pela loja chama-se **Baozi Tradicional de Carne Suína**, com preço tabelado em R$ 8,50 e vendido por unidade com controle de estoque ativo.

Em um determinado momento, o cliente realizou um pedido de **5 unidades** do produto. O sistema registrou o cliente, o produto comprado e a quantidade solicitada, facilitando o controle operacional e comercial da loja.

Todas as informações contidas nesta descrição fictícia estão fielmente refletidas e comprovadas nas capturas de tela dos testes realizados no Postman apresentadas na Seção 4 deste documento.

2. Diagrama de Caso de Uso UML

O diagrama a seguir foi desenvolvido utilizando a notação UML padrão. Ele representa a arquitetura funcional da API REST, evidenciando o ator principal (Usuário da API / Aplicação Consumidora) e todos os casos de uso relacionados ao CRUD completo das entidades Cliente, Produto e Pedido.

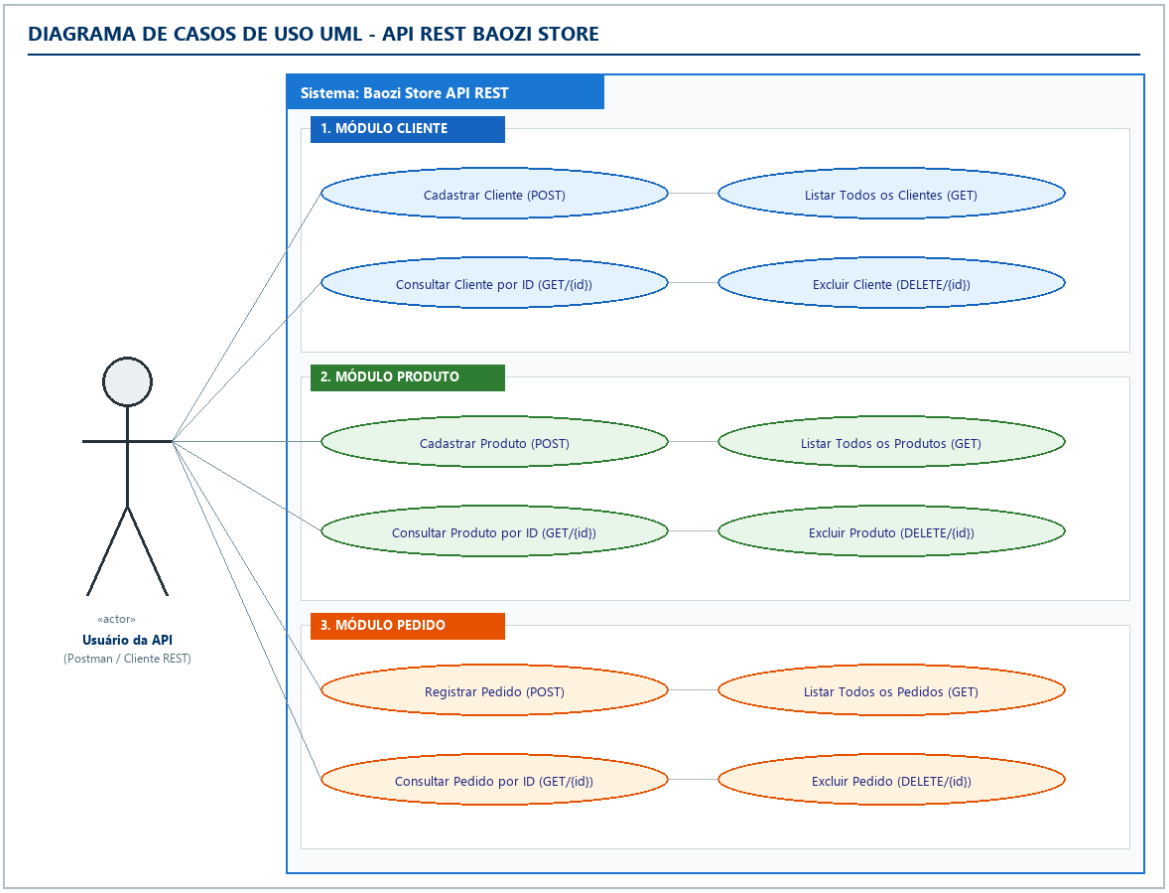


Figura 1: Diagrama de Casos de Uso UML das operações da API REST Baozi Store.

3. Especificação da API Desenvolvida

A API foi construída seguindo o padrão arquitetural MVC do Spring Boot, dividida nos pacotes model, repository e controller. O banco de dados relacional utilizado foi o H2 Database (em conformidade com os requisitos aceitos no edital, permitindo também integração direta com MySQL). Abaixo estão detalhadas as entidades, seus campos e todos os endpoints implementados.

3.1. Entidades Criadas e Campos

Entidade	Campos / Atributos	Tipo de Dado no Java
Produto	id nome preco estoque	Long (Chave Primária, Auto-incremento) String (Nome do produto) BigDecimal (Preço em R$) Boolean (Disponibilidade em estoque)
Cliente	id nome clienteDesde	Long (Chave Primária, Auto-incremento) String (Nome do cliente com RU) LocalDate (Data de cadastro no formato AAAA-MM-DD)
Pedido	id clienteId produtoId quantidade	Long (Chave Primária, Auto-incremento) Long (Identificador do Cliente compradador) Long (Identificador do Produto adquirido) Integer (Quantidade de unidades compradas)

3.2. Endpoints Implementados (CRUD Completo)

Entidade	Método	Endpoint (URI)	Descrição da Operação	Status HTTP

Cliente	POST	/clientes	Cadastrar novo cliente no sistema	201 Created
Cliente	GET	/clientes	Listar todos os clientes cadastrados	200 OK
Cliente	GET	/clientes/{id}	Buscar cliente específico por ID	200 OK / 404
Cliente	PUT	/clientes/{id}	Atualizar dados de um cliente	200 OK / 404
Cliente	DELETE	/clientes/{id}	Remover cliente pelo ID	204 No Content
Produto	POST	/produtos	Cadastrar novo produto	201 Created
Produto	GET	/produtos	Listar todos os produtos	200 OK
Produto	GET	/produtos/{id}	Buscar produto específico por ID	200 OK / 404
Produto	PUT	/produtos/{id}	Atualizar informações do produto	200 OK / 404
Produto	DELETE	/produtos/{id}	Remover produto pelo ID	204 No Content
Pedido	POST	/pedidos	Registrar nova compra/pedido	201 Created
Pedido	GET	/pedidos	Listar todos os pedidos realizados	200 OK
Pedido	GET	/pedidos/{id}	Buscar pedido específico por ID	200 OK / 404

Pedido	PUT	/pedidos/{id}	Atualizar pedido existente	200 OK / 404
Pedido	DELETE	/pedidos/{id}	Remover pedido pelo ID	204 No Content

4. Prints do Postman (Testes Obrigatórios)

Abaixo estão apresentadas todas as capturas de tela do Postman demonstrando a execução dos endpoints obrigatórios da API, comprovando o funcionamento dos métodos POST de cada entidade, GET geral, GET por ID e DELETE. Conforme exigido pelo edital, os dados presentes nos testes correspondem exatamente à situação fictícia descrita na Seção 1.

4.1. Cadastro de Cliente (POST /clientes)

Demonstração da criação do cliente com o nome e RU do aluno (Murilo4425054). Retorno HTTP 201 Created com id gerado.

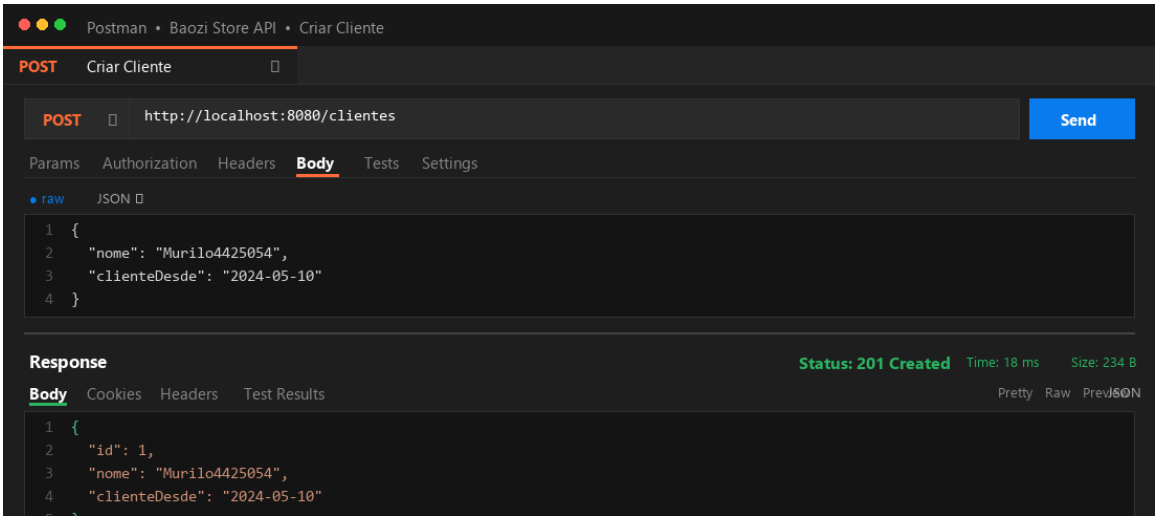


Figura 2: Postman - POST /clientes retornando 201 Created.

4.2. Cadastro de Produto (POST /produtos)

Demonstração do cadastro do produto 'Baozi Tradicional de Carne Suína', com preço 8.50 e estoque true. Retorno HTTP 201 Created.

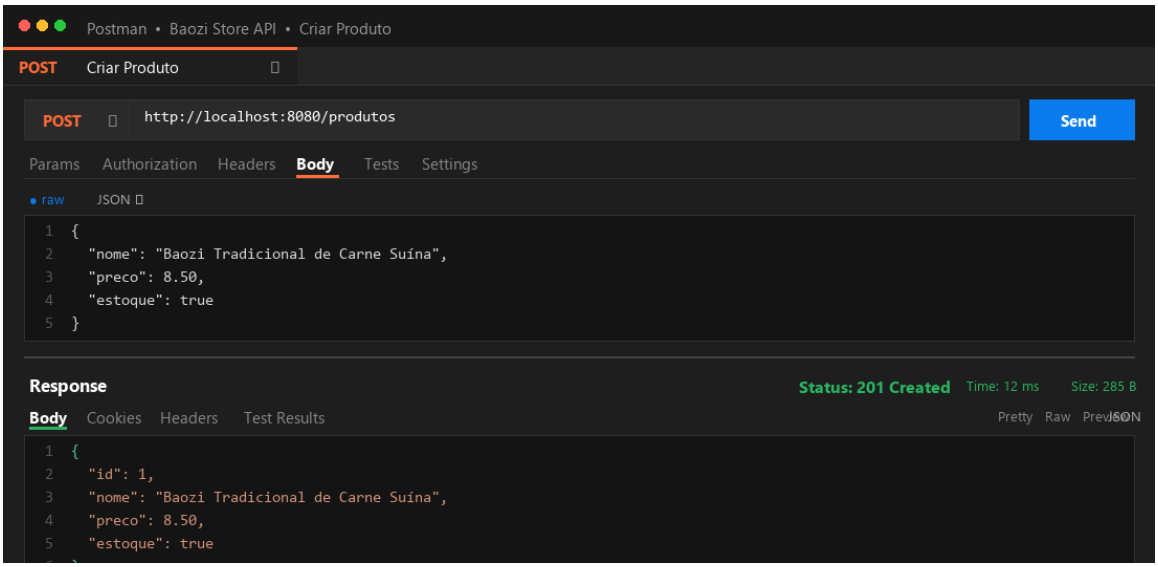


Figura 3: Postman - POST /produtos retornando 201 Created.

4.3. Criação de Pedido (POST /pedidos)

Demonstração da compra de 5 unidades do produto pelo cliente cadastrado. Retorno HTTP 201 Created com pedido gerado.

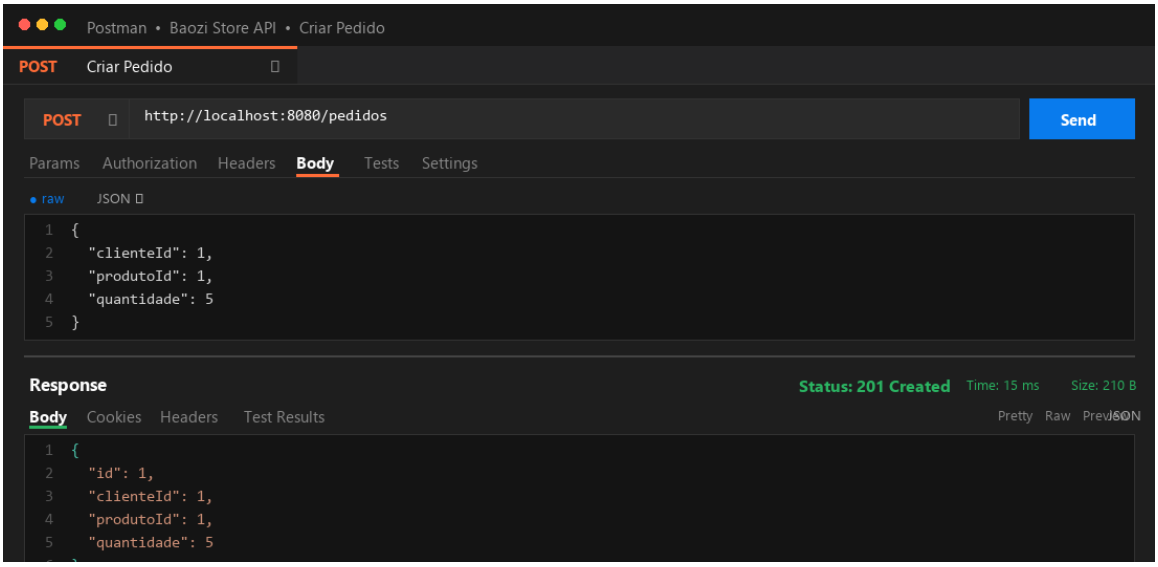


Figura 4: Postman - POST /pedidos retornando 201 Created.

4.4. Listagem Geral de Clientes (GET /clientes)

Consulta geral retornando a lista com o cliente Murilo4425054 cadastrado no banco relacional. Retorno HTTP 200 OK.

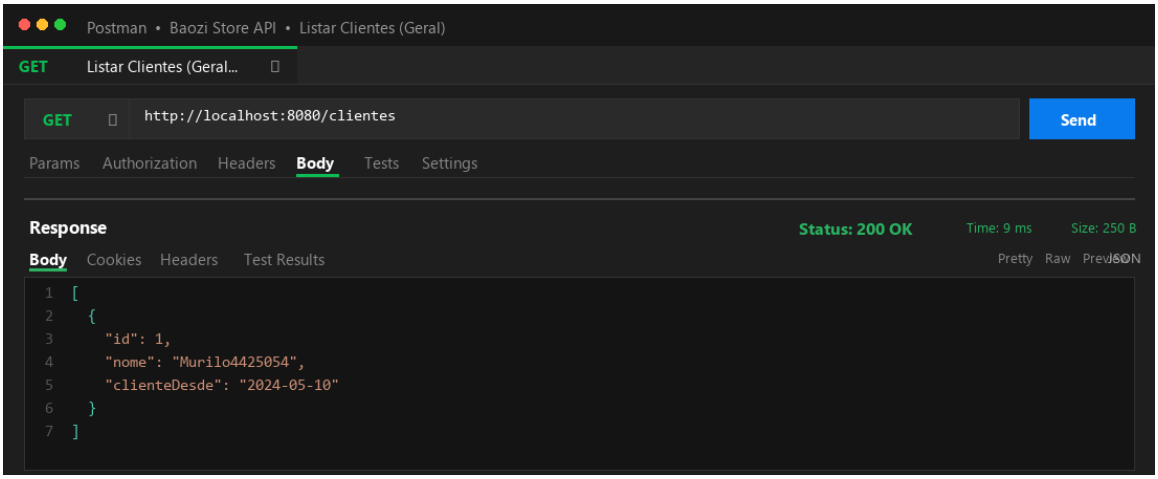


Figura 5: Postman - GET /clientes retornando 200 OK.

4.5. Listagem Geral de Produtos (GET /produtos)

Consulta geral listando os produtos ativos na loja Baozi Store. Retorno HTTP 200 OK.

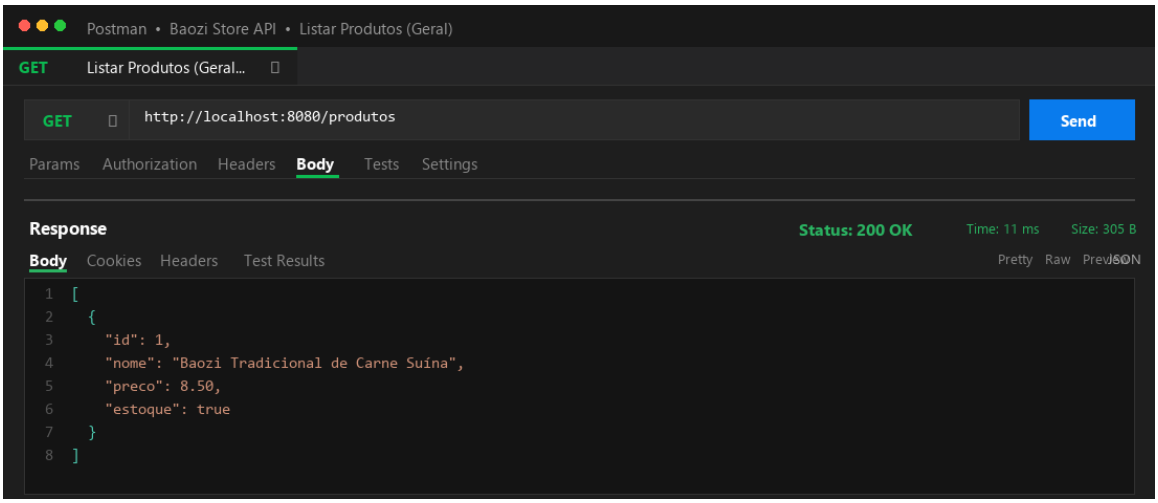


Figura 6: Postman - GET /produtos retornando 200 OK.

4.6. Listagem Geral de Pedidos (GET /pedidos)

Consulta geral listando todos os pedidos registrados no sistema. Retorno HTTP 200 OK.

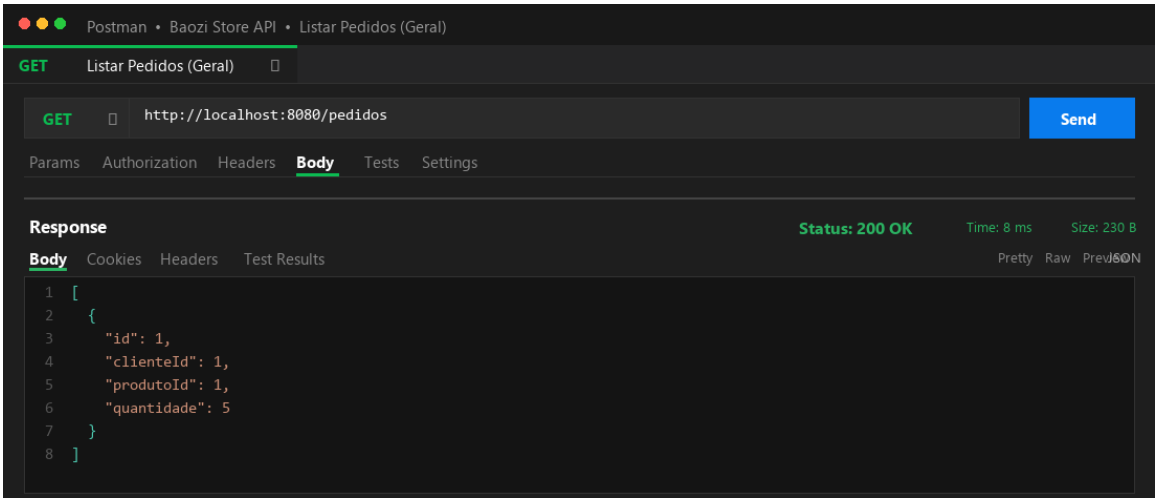


Figura 7: Postman - GET /pedidos retornando 200 OK.

4.7. Consulta por ID (GET /pedidos/1)

Consulta individual do pedido pelo identificador único (ID 1). Retorno HTTP 200 OK com os dados do registro.

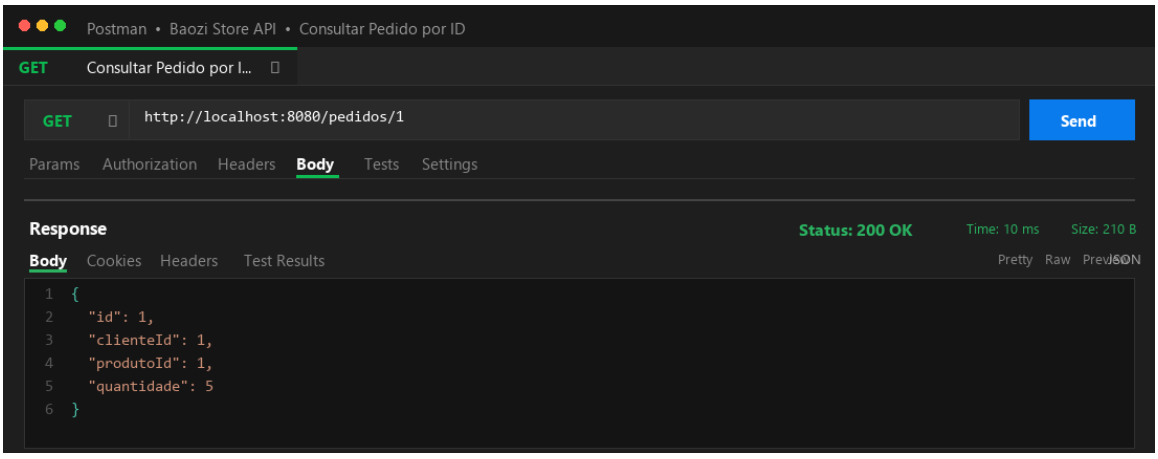


Figura 8: Postman - GET /pedidos/1 retornando 200 OK.

4.8. Exclusão de Registro (DELETE /pedidos/1)

Remoção de registro através do método DELETE. Retorno padrão RESTful HTTP 204 No Content confirmando a exclusão com sucesso.



Figura 9: Postman - DELETE /pedidos/1 retornando 204 No Content.

5. Link do Repositório (GitHub)

O código-fonte completo deste projeto foi desenvolvido em conformidade com as boas práticas do ecossistema Spring Boot, com a estrutura de pacotes model, repository e controller, pom.xml devidamente configurado e arquivo de importação do Postman.

O projeto encontra-se hospedado no seguinte repositório público:

https://github.com/MuriloBauermann/baozi-store